LMS - Learning Management System

SMK Telekomunikasi Tunas Harapan

Complete Setup Guide and Technical Documentation

Version 1.0 - Production Ready

Component	Technology
Backend Framework	Laravel 11 (PHP 8.2+)
Frontend	Blade + TailwindCSS 3
Database	MySQL 8.0+
Authentication	Laravel Breeze + Sanctum
Authorization	Spatie Laravel Permission
File Storage	Local + S3 Compatible
API	RESTful API (JSON)

Generated: March 2026

Table of Contents

1. Prerequisites and System Requirements

Before installing the LMS application, ensure that your server or development environment meets the following minimum requirements. These prerequisites are essential for the application to function correctly and provide optimal performance for all users, including administrators, teachers, and students accessing the platform simultaneously.

Server Requirements

Requirement	Minimum	Recommended
PHP	8.2	8.3+
MySQL	5.7	8.0+
Web Server	Apache 2.4+	Nginx 1.20+
RAM	512 MB	2 GB+
Disk Space	1 GB	10 GB+
Composer	2.6+	2.7+
Node.js/NPM	18+	20 LTS+

Required PHP Extensions

Laravel requires several PHP extensions to be installed and enabled. These extensions provide essential functionality for database connectivity, file processing, security, and various framework features. Ensure the following extensions are enabled in your php.ini configuration file before proceeding with installation.

- BCMath
- Ctype
- cURL
- DOM
- Fileinfo
- JSON
- Mbstring
- OpenSSL
- PDO
- Tokenizer
- XML

- GD
- ZIP
- MySQLi

2. Installation Step by Step

Follow these steps carefully to install and configure the LMS application on your server. The installation process involves cloning the project files, installing dependencies via Composer, configuring the environment, and setting up the database. Each step must be completed in order for the system to work properly.

Step 1: Copy Project Files

Copy the entire project directory to your web server document root or your preferred development directory. For a production server, the recommended path is /var/www/html/lms-smk-tunas-harapan. Ensure the web server user (www-data for Apache/Nginx) has appropriate read and write permissions for the storage and bootstrap/cache directories.

```
cp -r lms-smk-tunas-harapan /var/www/html/
cd /var/www/html/lms-smk-tunas-harapan
```

Step 2: Install Dependencies

Run Composer to install all PHP dependencies defined in the composer.json file. This includes the Laravel framework, Spatie permission package, Maatwebsite Excel for CSV export, Intervention Image for image processing, DomPDF for PDF generation, and other required libraries. The --optimize-autoloader flag improves class loading performance in production.

```
composer install --optimize-autoloader
```

Step 3: Set Directory Permissions

The storage and bootstrap/cache directories must be writable by the web server for Laravel to function properly. These directories store session files, compiled views, cache data, log files, and uploaded content. Setting the correct permissions prevents common 500 Internal Server Error issues.

```
chmod -R 775 storage bootstrap/cache
chown -R www-data:www-data storage bootstrap/cache
```

Step 4: Generate Application Key

Generate a unique application key that is used for encrypting session data, cookies, and other sensitive information. This key is stored in your .env file and must be kept secure. Never share or commit your .env file to version control.

```
php artisan key:generate
```

Step 5: Create Storage Link

Create a symbolic link from the public/storage directory to the storage/app/public directory. This makes uploaded files accessible from the web. Without this link, file downloads and image displays will not work properly.

```
php artisan storage:link
```

3. Database Configuration

The LMS application uses MySQL as its primary database. You need to create a MySQL database and user, then configure the connection settings in the .env file. After configuration, run the migrations to create all required tables and the seeders to populate the database with sample data for testing and demonstration purposes.

Step 1: Create MySQL Database

Log into your MySQL server and create a new database with UTF-8MB4 character set to support full Unicode including emoji characters. The UTF-8MB4 character set is recommended over the standard UTF-8 as it provides complete Unicode coverage, which is important for an educational platform that may need to display special characters and mathematical symbols in learning materials.

```
CREATE DATABASE lms_smk_tunas_harapan CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
CREATE USER 'lms_user'@'localhost' IDENTIFIED BY 'your_secure_password';
GRANT ALL PRIVILEGES ON lms_smk_tunas_harapan.* TO 'lms_user'@'localhost';
FLUSH PRIVILEGES;
```

Step 2: Update .env Configuration

Open the .env file in the project root and update the database connection settings to match your MySQL configuration. The DB_HOST should be localhost for local installations or the IP address/hostname for remote database servers. Make sure the DB_DATABASE, DB_USERNAME, and DB_PASSWORD values are correct.

```
DB_DATABASE=lms_smk_tunas_harapan
```

```
DB_USERNAME=lms_user
DB_PASSWORD=your_secure_password
```

Step 3: Run Migrations and Seeders

Execute the database migrations to create all 23 tables required by the LMS application. Then run the database seeders to populate the tables with sample data including user accounts, classes, subjects, materials, assignments, and more. This sample data is invaluable for testing the system and understanding its features before deploying to production.

```
php artisan migrate
php artisan db:seed
```

Alternatively, you can use the built-in artisan command that performs all setup steps at once including migration, seeding, key generation, and storage link creation:

```
php artisan lms:install
```

4. Environment Setup

The .env file contains all environment-specific configuration for your application. Properly configuring these settings is critical for security, email delivery, file uploads, and overall application behavior. Review and update each setting according to your deployment environment, whether it is local development, staging, or production.

Key Configuration Settings

Setting	Description	Example
APP_NAME	Application display name	LMS SMK Tunas Harapan
APP_ENV	Environment mode	production
APP_URL	Application base URL	https://lms.smktunas.sch.id
MAIL_MAILER	Email driver	smtp
MAIL_HOST	SMTP server	smtp.gmail.com
AWS_*	S3 storage credentials	(optional)

5. Running the Application

Development Server

For local development and testing, you can use the built-in PHP development server that comes with Laravel. This is the quickest way to get the application running without configuring a full web server. The development server provides hot-reloading capabilities and displays detailed error messages that help with debugging.

```
php artisan serve
```

The application will be accessible at http://localhost:8000. You can specify a different host and port if needed by adding the --host and --port options to the serve command.

Production Deployment

For production deployment, configure your web server (Apache or Nginx) to point to the public directory of the project. The document root must be set to the public directory, not the project root, for security reasons. This ensures that only the public files are accessible via the web, while application code remains protected from direct access.

After deploying to production, run the following optimization commands to improve performance by caching configuration, routes, views, and events. These commands significantly reduce the number of file system operations on each request, resulting in faster page load times and reduced server load.

```
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan event:cache
php artisan optimize
```

6. Default Login Credentials

After running the database seeders, the following user accounts are created for testing purposes. These accounts provide access to different user roles within the system, allowing you to test all features and functionality. In a production environment, you should change all default passwords immediately after deployment and remove or disable any test accounts that are not needed.

Role	Email	Password	Name
Admin	admin@smktunas.sch.id	password	Administrator
Guru 1	guru1@smktunas.sch.id	password123	Budi Santoso, S.Kom
Guru 2	guru2@smktunas.sch.id	password123	Siti Nurhaliza, S.Pd
Siswa 1	siswa1@smktunas.sch.id	password123	Ahmad Fauzi
Siswa 2	siswa2@smktunas.sch.id	password123	Dewi Lestari

7. System Architecture Overview

The LMS application follows the Model-View-Controller (MVC) architectural pattern, which is the standard approach for Laravel applications. This separation of concerns ensures that business logic, data access, and presentation layers are cleanly separated, making the codebase maintainable, testable, and scalable as the system grows.

User Roles and Permissions

The system implements a role-based access control (RBAC) system using the Spatie Laravel Permission package. Each user is assigned one or more roles, and each role can have multiple permissions. This granular permission system allows fine-grained control over what each user can access and do within the application. The three primary roles in the LMS are Administrator, Guru (Teacher), and Siswa (Student), each with distinct capabilities.

Feature	Admin	Guru	Siswa
Manage Users	Full CRUD	No	No
Manage Classes	Full CRUD	Create/Own	Join/View
Manage Subjects	Full CRUD	View	View
Create Materials	Yes	Yes (Own)	No
Create Assignments	Yes	Yes (Own)	No

Grade Submissions	Yes	Yes (Own)	No
Submit Assignments	No	No	Yes
View Grades	All	Own Classes	Own
Attendance	View All	Manage	View Own
System Settings	Full Access	No	No

8. Database Schema

The LMS database consists of 23 interconnected tables that store all application data. The schema is designed with proper normalization, foreign key constraints, and performance-optimized indexes. The tables cover user management, academic data (classes, subjects, materials, assignments), assessment (submissions, grades, quizzes), communication (comments, notifications, announcements), and system administration (activity logs, attendance).

Table	Description	Key Fields
users	All user accounts (admin, guru, siswa)	name, email, role, NIP/NIS
roles	System roles	name (admin/guru/siswa)
jurusans	Study programs	kode, nama (PPLG, TJKT)
tahun_ajarans	Academic years	nama, semester, aktif
mapels	Subjects	nama, kategori, KKM
kelas	Classes	nama, kode_unik, guru_id
materis	Learning materials	judul, tipe, konten
tugas	Assignments	judul, deadline, nilai_maks
submissions	Student submissions	konten, status, nilai
comments	Polymorphic comments	body, parent_id, type
notifications	User notifications	title, is_read, link
attendances	Attendance sessions	tanggal, pertemuan_ke
quizzes	Online quizzes	durasi, random_soal
quiz_questions	Quiz questions	pertanyaan, tipe, poin
quiz_attempts	Student quiz attempts	skor, answers (JSON)

9. Module Descriptions

Authentication and Role System

Multi-role authentication supporting Admin, Guru, and Siswa. Features include role-based middleware, login/logout, password reset via email, and session management. Admin users can create and manage all accounts while regular users can update their own profiles. The system uses Laravel Breeze for web authentication and Sanctum for API token-based authentication.

Dashboard

Role-specific dashboards provide each user type with relevant information at a glance. Admin dashboard shows system-wide analytics, user counts, and activity logs. Guru dashboard displays assigned classes, pending grading tasks, upcoming deadlines, and attendance summaries. Siswa dashboard shows enrolled classes, upcoming assignments, recent grades, and notifications.

Class Management (Kelas)

Guru can create classes with unique join codes. Siswa can join classes by entering the code. Each class has a dedicated page with a timeline feed similar to Google Classroom. Classes are organized by jurusan (PPLG, TJKT), tingkat (X, XI, XII), and tahun ajaran. Admin manages class creation, wali kelas assignment, and student enrollment.

Learning Materials (Materi)

Guru can create rich learning content supporting multiple formats: PDF file uploads, YouTube video embeds, rich text content, and external links. Materials are organized by mata pelajaran (subject) and pertemuan (meeting number). Published materials are visible to enrolled siswa, while draft materials remain private.

Assignments (Tugas)

Comprehensive assignment management with deadline tracking, file attachments, and multiple assignment types (tugas, proyek, quiz, ujian). Students can submit text answers and/or file uploads. The system automatically tracks submission status (submitted, late, not submitted) and notifies students of approaching deadlines.

Grading (Penilaian)

Guru can grade individual submissions with numeric scores and text feedback. Bulk grading allows efficient assessment of entire classes. The system automatically calculates pass/fail status based on KKM (Kriteria Ketuntasan Minimal). Grade recaps show student performance across all assignments, and results can be exported to CSV for external analysis.

Discussion Forum

Threaded commenting system on materials and assignments supports nested replies, user mentions, and polymorphic associations. Both guru and siswa can participate in discussions, fostering collaborative learning. The system tracks edit history and provides appropriate moderation controls.

Notifications

Real-time notification system keeps users informed about new assignments, approaching deadlines, grade releases, and announcements. The system uses AJAX polling to update notification counts in the navbar, and provides a dedicated notification management page with filtering and mark-as-read functionality.

Attendance (Absensi)

Guru can record daily student attendance with support for hadir (present), izin (permitted absence), sakit (sick leave), and alpha (unexcused absence) statuses. The system generates attendance recaps per student and class, calculates attendance percentages, and allows export to CSV format.

Online Quiz System

Full-featured quiz engine with support for multiple question types (multiple choice, true/false, essay). Features include timed quizzes, random question ordering, automatic scoring for objective questions, and detailed result reviews with explanations. Guru can view all attempts and performance analytics.

10. API Endpoints Reference

The LMS exposes a comprehensive RESTful API for integration with external applications, mobile apps, and third-party services. All API endpoints use JSON format for request and response bodies, and authenticated endpoints require a Sanctum API token passed via the Authorization Bearer header. Rate limiting is applied to prevent abuse.

Method	Endpoint	Description
POST	/api/login	Authenticate user, get token
POST	/api/logout	Revoke authentication token
GET	/api/me	Get current user info
GET	/api/kelas	List classes
POST	/api/kelas	Create class (guru)
POST	/api/kelas/join	Join class via code (siswa)
GET	/api/kelas/{id}/materi	List class materials
POST	/api/kelas/{id}/materi	Create material
GET	/api/kelas/{id}/tugas	List class assignments
POST	/api/tugas/{id}/submissions	Submit assignment
POST	/api/submissions/{id}/grade	Grade submission (guru)
GET	/api/notifications	List notifications
POST	/api/notifications/{id}/read	Mark notification as read
GET	/api/notifications/unread-count	Get unread notification count

11. Security Features

Security is a top priority in the LMS application. The system implements multiple layers of protection to safeguard user data, prevent unauthorized access, and ensure the integrity of academic records. The following security measures are built into the application by default, following Laravel security best practices and OWASP guidelines.

CSRF Protection

All forms include Laravel CSRF tokens to prevent Cross-Site Request Forgery attacks. The VerifyCsrf middleware automatically validates tokens on all POST, PUT, PATCH, and DELETE requests, rejecting any request with an invalid or missing token.

Input Validation

All user inputs are validated using Laravel validation rules before processing. This prevents SQL injection, XSS attacks, and malformed data from entering the system. Validation rules include required fields, email format, file types, and size limits.

File Upload Security

Uploaded files are validated for MIME type, file size, and extension. The system restricts uploads to safe file types (PDF, DOCX, images, common video formats) and stores files outside the public directory for enhanced security.

Password Security

Passwords are hashed using bcrypt with a cost factor of 12. The system enforces strong password requirements including minimum length, complexity rules, and validation against common passwords using Laravel Password broker.

Role-Based Authorization

Access to routes and features is controlled through role-based middleware. Each user role has specific permissions, and unauthorized access attempts are blocked with appropriate 403 error responses.

SQL Injection Prevention

Laravel Eloquent ORM uses parameter binding for all database queries, effectively preventing SQL injection attacks. Raw SQL queries, when necessary, use parameterized statements exclusively.

XSS Protection

Blade templates automatically escape output by default, preventing Cross-Site Scripting attacks. Rich text content is sanitized before storage and display to remove potentially harmful HTML and JavaScript.

Rate Limiting

API and login routes are rate-limited to prevent brute-force attacks and abuse. Failed login attempts are tracked and throttled after a configurable number of failures.

12. File Structure Reference

The following table provides a comprehensive overview of the project file structure, listing the number of files in each major directory and their purpose. This reference helps developers navigate the codebase efficiently and understand the organizational structure of the application.

Directory	Files	Purpose
app/Models/	20	Eloquent ORM models with relationships
app/Http/Controllers/Admin/	8	Admin panel controllers
app/Http/Controllers/Guru/	8	Teacher dashboard controllers
app/Http/Controllers/Siswa/	8	Student dashboard controllers
app/Http/Controllers/Api/	8	RESTful API controllers
app/Http/Controllers/Auth/	3	Authentication controllers
app/Http/Middleware/	4	Role-check and security middleware
app/Enums/	4	PHP 8.1 backed enumerations
app/Providers/	4	Service providers
app/Helpers/	1	Global helper functions
database/migrations/	23	Database schema migrations
database/seeders/	16	Database seeders with sample data
resources/views/	91	Blade templates (layouts, pages)
config/	8	Application configuration files
routes/	3	Route definitions (web, api, console)

Total: 221 files across 40+ directories

13. Troubleshooting Common Issues

500 Internal Server Error

This is typically caused by incorrect file permissions or a missing .env file. Ensure the storage and bootstrap/cache directories have 775 permissions and are owned by the web server user. Also verify that the .env file exists and contains valid configuration. Run "php artisan config:clear" to clear any stale cached configuration.

Database Connection Error

Verify that your MySQL server is running and the credentials in the .env file are correct. Check that the database exists and the user has proper privileges. Try running "php artisan migrate:status" to test the connection. Ensure the DB_PORT is set correctly (default is 3306).

Page Not Found (404)

After deploying, run "php artisan route:clear" and "php artisan config:cache" to refresh route caching. For Apache, verify that mod_rewrite is enabled and the .htaccess file exists in the public directory. For Nginx, check that the try_files directive is properly configured.

File Upload Not Working

Ensure the storage link has been created with "php artisan storage:link". Verify that the storage/app/public directory has proper write permissions. Check that the uploaded file type is allowed in the validation rules and does not exceed the configured maximum file size in php.ini (upload_max_filesize and post_max_size).

Session Issues

If users are being logged out unexpectedly, check that the SESSION_DRIVER in .env is set appropriately. For production, use "database" or "redis" instead of "file". Also verify that the storage/framework/sessions directory is writable.

Email Not Sending

Verify the MAIL_* settings in your .env file. For Gmail, you need to enable "Less secure apps" or use an App Password with 2FA enabled. Test the mail configuration with "php artisan tinker" and then running "Mail::raw("Test", function($msg) { $msg->to("test@example.com")->subject("Test"); });".